```
import uvm_pkg::*;
import ALSU_test_pkg::*;
`include "uvm_macros.svh"
module ALSU_top();

    bit clk = 0;

    initial begin
        forever #1 clk = ~clk;
    end

    ALSU_if ALSUif(clk);

    ALSU  #(.INPUT_PRIORITY(ALSUif.INPUT_PRIORITY),
            .FULL_ADDER(ALSUif.FULL_ADDER))
          DUT (ALSUif.A,
           ALSUif.B,
           ALSUif.cin,
           ALSUif.serial_in,
           ALSUif.red_op_A,
           ALSUif.red_op_B,
           ALSUif.opcode,
           ALSUif.bypass_A,
           ALSUif.bypass_B,
           ALSUif.clk,
           ALSUif.rst,
           ALSUif.direction,
           ALSUif.leds,
           ALSUif.out);

    ALSU_sva ALSU_sva_inst(ALSUif);

    initial begin
        uvm_config_db#(virtual ALSU_if)::set(null,"uvm_test_top","ALSU_IF",ALSUif);
        run_test("ALSU_test");
    end
endmodule
```

```
interface ALSU_if(input logic clk);

    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";

    logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    logic [2:0] opcode;
    logic signed [2:0] A, B;
    logic [15:0] leds;
    logic signed [5:0] out;

endinterface //interfacename
```

```
package ALSU_config_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

    class ALSU_config extends uvm_object;
        `uvm_object_utils(ALSU_config)

        virtual ALSU_if ALSU_vif;

    function new(string name = "ALSU_config");
            super.new(name);
    endfunction

    endclass

endpackage
```

```
package ALSU_test_pkg;
import uvm_pkg::*;
import ALSU_env_pkg::*;
import ALSU_config_pkg::*;
import ALSU_sequence_pkg::*;
`include "uvm_macros.svh"

    class ALSU_test extends uvm_test;
        `uvm_component_utils(ALSU_test)

        virtual ALSU_if ALSUif;
        ALSU_env env;
        ALSU_config ALSU_cfg;
        ALSU_reset_sequence reset_seq;
        ALSU_main_sequence main_seq;

        function new(string name = "ALSU_test",uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = ALSU_env::type_id::create("env",this);
            ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg");
            reset_seq =  ALSU_reset_sequence::type_id::create("reset_seq",this);
            main_seq =  ALSU_main_sequence::type_id::create("main_seq",this);

            if (!uvm_config_db#(virtual ALSU_if)::get(this,"","ALSU_IF",ALSU_cfg.ALSU_vif))
                `uvm_fatal("build_phase","Test- unable to get the virtual interface");

            uvm_config_db#(ALSU_config)::set(this,"*","CFG",ALSU_cfg);
        endfunction
```

```
        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            // reset sequence
            phase.raise_objection(this);
            `uvm_info("run_phase","Reset asserted", UVM_LOW)
            reset_seq.start(env.agt.sqr);
            `uvm_info("run_phase","Reset deasserted", UVM_LOW)

            // main sequence
            `uvm_info("run_phase","Stimulus generation started", UVM_LOW)
            main_seq.start(env.agt.sqr);
            `uvm_info("run_phase","Stimulus generation ended", UVM_LOW)
            #10;
            phase.drop_objection(this);

        endtask

    endclass


endpackage
```

```
package ALSU_sequence_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
import ALSU_pkg::*;
`include "uvm_macros.svh"

    class ALSU_reset_sequence extends uvm_sequence #(ALSU_seq_item);
        `uvm_object_utils(ALSU_reset_sequence)
        ALSU_seq_item seq_item;

        function new(string name = "ALSU_reset_sequence");
            super.new(name);
        endfunction

        task body();
            seq_item = ALSU_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            seq_item.rst = 1;
            seq_item.cin = 0;
            seq_item.red_op_A = 0;
            seq_item.red_op_B = 0;
            seq_item.bypass_A = 0;
            seq_item.bypass_B = 0;
            seq_item.direction = 0;
            seq_item.serial_in = 0;
            seq_item.opcode = opcode_e'(0);
            seq_item.A = 0;
            seq_item.B = 0;
            finish_item(seq_item);
        endtask

    endclass
```

```
class ALSU_main_sequence extends uvm_sequence #(ALSU_seq_item);
    `uvm_object_utils(ALSU_main_sequence)
    ALSU_seq_item seq_item;

    function new (string name = "ALSU_main_sequence");
        super.new(name);
    endfunction

    task body();
        seq_item = ALSU_seq_item::type_id::create("seq_item");

        // TASK 1: constraint A only
        seq_item.opcode_cst.constraint_mode(1);
        seq_item.opcode_seq_con.constraint_mode(0);
        repeat(1000) begin
            start_item(seq_item);
            assert(seq_item.randomize());
            finish_item(seq_item);
        end

        // TASK 2: constraint B only
        seq_item.opcode_cst.constraint_mode(0);
        seq_item.opcode_seq_con.constraint_mode(1);
        repeat (1000) begin
            assert(seq_item.randomize() with {seq_item.rst == 0;});
            for (int i = 0; i < 6; i++) begin
                start_item(seq_item);
                seq_item.opcode = seq_item.opcode_valid[i];
                finish_item(seq_item);
            end

        end
    endtask
endclass
endpackage
```

```
package ALSU_env_pkg;
import uvm_pkg::*;
import ALSU_agent_pkg::*;
import ALSU_scoreboard_pkg::*;
import ALSU_coverage_pkg::*;
`include "uvm_macros.svh"

    class ALSU_env extends uvm_env;
        `uvm_component_utils(ALSU_env)

        ALSU_agent agt;
        ALSU_scoreboard sb;
        ALSU_coverage cov;

        function new(string name = "ALSU_env", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);

            agt = ALSU_agent::type_id::create("agt",this);
            sb = ALSU_scoreboard::type_id::create("sb",this);
            cov = ALSU_coverage::type_id::create("cov",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            /// connect agent with scoreboard and coverage
            agt.agt_ap.connect(sb.sb_export);
            agt.agt_ap.connect(cov.cov_export);
        endfunction

    endclass

endpackage
```

```
package ALSU_agent_pkg;
import uvm_pkg::*;
import ALSU_monitor_pkg::*;
import ALSU_driver_pkg::*;
import ALSU_seq_item_pkg::*;
import ALSU_sequencer_pkg::*;
import ALSU_config_pkg::*;
`include "uvm_macros.svh"

    class ALSU_agent extends uvm_agent;
        `uvm_component_utils(ALSU_agent)

        ALSU_driver drv;
        ALSU_monitor mon;
        ALSU_config ALSU_cfg;
        ALSU_sequencer sqr;
        uvm_analysis_port #(ALSU_seq_item) agt_ap;

        function new(string name = "ALSU_agent", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);

            if(!uvm_config_db#(ALSU_config)::get(this,"","CFG",ALSU_cfg))
                `uvm_fatal("build phase","Agent - Unable to get configuration object");

            drv = ALSU_driver::type_id::create("drv",this);
            mon = ALSU_monitor::type_id::create("mon",this);
            sqr = ALSU_sequencer::type_id::create("sqr",this);
            agt_ap = new("agt_ap",this);

        endfunction
```

```
        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            drv.ALSU_vif = ALSU_cfg.ALSU_vif;
            mon.ALSU_vif = ALSU_cfg.ALSU_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
            mon.mon_ap.connect(agt_ap);
        endfunction

    endclass

endpackage
```

```
package ALSU_monitor_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
import ALSU_pkg::*;
`include "uvm_macros.svh"

    class ALSU_monitor extends uvm_monitor;
        `uvm_component_utils(ALSU_monitor)

        virtual ALSU_if ALSU_vif;
        ALSU_seq_item rsp_seq_item;
        uvm_analysis_port #(ALSU_seq_item) mon_ap;

        function new(string name = "ALSU_monitor", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon_ap = new("mon_ap",this);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            rsp_seq_item = ALSU_seq_item::type_id::create("rsp_seq_item",this);
            forever begin
                @(negedge ALSU_vif.clk);
                rsp_seq_item.rst = ALSU_vif.rst;
                rsp_seq_item.cin = ALSU_vif.cin;
                rsp_seq_item.red_op_A = ALSU_vif.red_op_A;
                rsp_seq_item.red_op_B = ALSU_vif.red_op_B;
                rsp_seq_item.bypass_A = ALSU_vif.bypass_A;
                rsp_seq_item.bypass_B = ALSU_vif.bypass_B;
                rsp_seq_item.direction = ALSU_vif.direction;
                rsp_seq_item.serial_in = ALSU_vif.serial_in;
                rsp_seq_item.opcode = opcode_e'(ALSU_vif.opcode);
                rsp_seq_item.A = ALSU_vif.A;
                rsp_seq_item.B = ALSU_vif.B;
                mon_ap.write(rsp_seq_item); //broadcast
                `uvm_info("run_phase",rsp_seq_item.convert2string_stimulus(),UVM_HIGH);
            end
        endtask

    endclass

endpackage
```

```
package ALSU_driver_pkg;
import uvm_pkg::*;
import ALSU_config_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"
    class ALSU_driver extends uvm_driver #(ALSU_seq_item);
        `uvm_component_utils(ALSU_driver)

        virtual ALSU_if ALSU_vif;
        ALSU_seq_item stim_seq_item;

        function new(string name = "ALSU_driver", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            stim_seq_item = ALSU_seq_item::type_id::create("stim_seq_item",this);
            forever begin
                seq_item_port.get_next_item(stim_seq_item);
                ALSU_vif.rst = stim_seq_item.rst;
                ALSU_vif.cin = stim_seq_item.cin;
                ALSU_vif.red_op_A = stim_seq_item.red_op_A;
                ALSU_vif.red_op_B = stim_seq_item.red_op_B;
                ALSU_vif.bypass_A = stim_seq_item.bypass_A;
                ALSU_vif.bypass_B = stim_seq_item.bypass_B;
                ALSU_vif.direction = stim_seq_item.direction;
                ALSU_vif.serial_in = stim_seq_item.serial_in;
                ALSU_vif.opcode = stim_seq_item.opcode;
                ALSU_vif.A = stim_seq_item.A;
                ALSU_vif.B = stim_seq_item.B;
                @(negedge ALSU_vif.clk);
                seq_item_port.item_done();
            end
        endtask
    endclass
endpackage
```

```
package ALSU_seq_item_pkg;
import uvm_pkg::*;
import ALSU_pkg::*;
`include "uvm_macros.svh"

    parameter VALID_OP = 6;

    class ALSU_seq_item extends uvm_sequence_item;
        `uvm_object_utils(ALSU_seq_item)

        rand bit rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
        rand opcode_e opcode;
        rand opcode_e opcode_valid[VALID_OP];
        rand bit signed [2:0] A;
        rand bit signed [2:0] B;
        bit [5:0] out;
        bit [15:0] leds;

        // Declare the random variables for inputs A and B, and their corner cases and walking ones patterns
        rand reg_e enum_ext_A, enum_ext_B;
        rand logic [2:0] A_rem_val, B_rem_val;

        bit [2:0] walking_ones[] = '{3'b100, 3'b010, 3'b001};
        rand bit [2:0] walking_ones_t, walking_ones_f;

        function new(string name = "ALSU_seq_item");
            super.new(name);
        endfunction

        function string convert2string();
            return $sformatf("%s reset=%0b cin=%0b red_op_A=%0b red_op_B=%0b bypass_A=%0b bypass_B=%0b direction=%0b serial_in=%0b opcode=%0b A=%0d B=%0d out=%0d leds=%0d
                             super.convert2string(),
                             rst, cin, red_op_A, red_op_B, bypass_A, bypass_B,
                             direction, serial_in, opcode, A, B, out, leds);
        endfunction
```

```
        function string convert2string();
            return $sformatf("%s reset=%0b cin=%0b red_op_A=%0b red_op_B=%0b bypass_A=%0b bypass_B=%0b direction=%0b serial_in=%0b opcode=%0b A=%0d B=%0d out=%0d leds=%0d
                             super.convert2string(),
                             rst, cin, red_op_A, red_op_B, bypass_A, bypass_B,
                             direction, serial_in, opcode, A, B, out, leds);
        endfunction

        function string convert2string_stimulus();
            return $sformatf("reset = 0b%0b, cin = 0b%0b,red_op_A = 0b%0b, red_op_B = 0b%0b, bypass_A = 0b%0b, bypass_B = 0b%0b, direction = 0b%0b , serial_in = 0b%0b , op
                             rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode, A, B,out, leds);
        endfunction


        // constraints
        //1. Reset to be asserted with a low probability that you decide.
        constraint rst_cst{
            rst dist{0:= 99, 1:= 1};
        }
```

```
        //2. The inputs A and B to be mostly zero, with some random values.
        constraint A_B_cst{
            walking_ones_t inside{walking_ones}; // Constrain the walking_ones_t to be one of the patterns in walking_ones
            !(walking_ones_f inside {walking_ones}); // Constrain the walking_ones_f to not be any of the patterns in walking_ones
            !(A_rem_val inside {enum_ext_A}); // Constrain A_rem_val to not be any of the corner cases
            !(B_rem_val inside {enum_ext_B}); // Constrain B_rem_val to not be any of the corner cases

            // If the opcode is OR or XOR
            if ((opcode == OR) || (opcode == XOR)){
                // red_op_A is high, constraint the input A most of the time
                // to have one bit high in its 3 bits while constraining the B bits to be low
                if (red_op_A == 1){
                    A dist {walking_ones_t := 90, walking_ones_f := 10};
                    B == 3'b000;
                }
                // red_op_B is high, constraint the input B most of the time
                // to have one bit high in its 3 bits while constraining the A bits to be low
                else if (red_op_B == 1){
                    B dist {walking_ones_t := 90, walking_ones_f := 10};
                    A == 3'b000;
                }
            } else {
                red_op_A dist {1:= 10, 0 := 90};
                red_op_B dist {1:= 10, 0 := 90};
                // If the opcode is ADD or MULT, constraint the inputs A and B  to take the values (MAXPOS, ZERO and MAXNEG) with higher probability than the other values
                if ((opcode == ADD) || (opcode == MULT)){
                A dist {enum_ext_A := 90, A_rem_val := 10};
                B dist {enum_ext_B := 90, B_rem_val := 10};
            }
            }
        }
```

```
    // 3. The opcode to be mostly OR, XOR, ADD, MULT, ROTATE, SHIFT, with some random values for the other operations.
    // will be used while turning off opcode_arr_cst to allow random values in the opcode_arr which will be used in the covergroup to cover the transitions between al
    constraint opcode_cst{
        opcode dist{[OR:ROTATE] := 90, [INVALID6:INVALID7] := 10};
    }

    // 4. The bypass signals to be mostly disabled, with some random values.
    constraint bypass_cst{
        bypass_A dist{0:= 95, 1:= 5};
        bypass_B dist{0:= 95, 1:= 5};
    }

    // Create a fixed array of 6 elements of type opcode_e. constraint the elements of the array using
    // foreach to have a unique valid value each time randomization occurs.
    // will be used in the covergroup to cover all the operations and the transitions between them while turning off opcode_cst to allow the array to be randomized wi
    constraint opcode_seq_con{
        foreach(opcode_valid[i])
            foreach(opcode_valid[j]){
                if(i != j){
                opcode_valid[i] != opcode_valid[j];
                opcode_valid[i] inside {[OR:ROTATE]};
                }
            }
    }
  endclass

endpackage
```

```
package ALSU_sequencer_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"

    class ALSU_sequencer extends uvm_sequencer#(ALSU_seq_item);
        `uvm_component_utils(ALSU_sequencer)

        function new(string name = "ALSU_sequencer",uvm_component parent = null);
            super.new(name,parent);
        endfunction

    endclass

endpackage
```

```
package ALSU_scoreboard_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"

    class ALSU_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(ALSU_scoreboard)

        uvm_analysis_export #(ALSU_seq_item) sb_export;
        uvm_tlm_analysis_fifo #(ALSU_seq_item) sb_fifo;
        ALSU_seq_item seq_item_sb;

        function new(string name = "ALSU_scoreboard", uvm_component parent = null);
            super.new(name,parent);
            sb_export = new("sb_export",this);
            sb_fifo = new("sb_fifo",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            sb_export.connect(sb_fifo.analysis_export);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                sb_fifo.get(seq_item_sb);
            end
        endtask

    endclass



endpackage
```

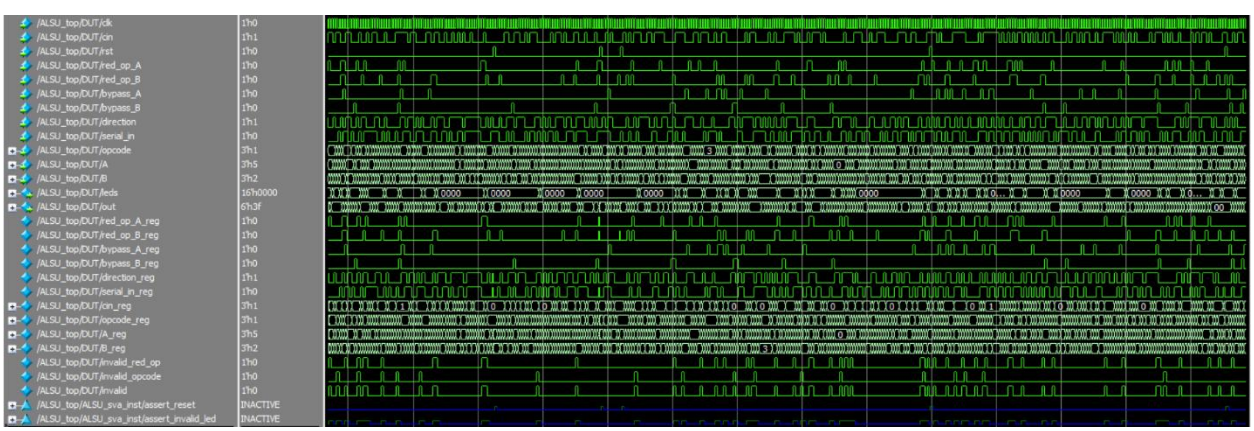
```
package ALSU_coverage_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
import ALSU_pkg::*;
`include "uvm_macros.svh"

        class ALSU_coverage extends uvm_component;
        `uvm_component_utils(ALSU_coverage)

        uvm_analysis_export #(ALSU_seq_item) cov_export;
        uvm_tlm_analysis_fifo #(ALSU_seq_item) cov_fifo;
        ALSU_seq_item seq_item_cov;


        //covergroup
        covergroup cvr_gp;

            // Cover A values (corner cases + misc)
            A_cp1: coverpoint seq_item_cov.A {
                bins A_data_0 = {3'b000};
                bins A_data_max = {3'b011};
                bins A_data_min = {3'b100};
                bins A_data_default = default;
            }

            // Walking ones when red_op_A = 1
            A_cp2: coverpoint seq_item_cov.A iff(seq_item_cov.red_op_A){
                bins A_data_walkingones[] ={3'b001, 3'b010, 3'b100};
            }

            // Cover B values (corner cases + misc)
            B_cp1: coverpoint seq_item_cov.B {
                bins B_data_0 = {3'b000};
                bins B_data_max = {3'b011};
                bins B_data_min = {3'b100};
                bins B_data_default = default;
            }
```

```
// Walking ones when red_op_B = 1
B_cp2: coverpoint seq_item_cov.B iff(seq_item_cov.red_op_B  && !seq_item_cov.red_op_A ) {
    bins B_data_walkingones[] ={3'b001, 3'b010, 3'b100};
}

// Cover the opcode with the specified bins and transitions
ALU_cp: coverpoint seq_item_cov.opcode{
    bins bins_shift[] = {SHIFT, ROTATE};
    bins bins_arith[] = {ADD, MULT};
    bins bins_bitwise[] = {OR, XOR};
    bins bins_invalid[] = {INVALID6, INVALID7};
    bins bins_trans = (OR => XOR => ADD => MULT => SHIFT => ROTATE);
}

opcode_cp: coverpoint seq_item_cov.opcode;

cin_cp: coverpoint seq_item_cov.cin {
    bins cin_zero = {0};
    bins cin_one = {1};
}

dir_cp: coverpoint seq_item_cov.direction{
    bins dir_zero = {0};
    bins dir_one = {1};
}

shift_cp: coverpoint seq_item_cov.serial_in{
    bins shift_zero = {0};
    bins shift_one = {1};
}

redA_cp: coverpoint seq_item_cov.red_op_A{
    bins redA_on = {1};
    bins redA_off = {0};
}
```

```
redB_cp: coverpoint seq_item_cov.red_op_B{
    bins redB_on = {1};
    bins redB_off = {0};
}

add_mult_AB_cs : cross opcode_cp, A_cp1, B_cp1{
    option.cross_auto_bin_max = 0;
    bins add_mult_A_B = binsof(opcode_cp) intersect{ADD,MULT} &&
                        binsof(A_cp1) intersect{MAXPOS, MAXNEG, ZERO} &&
                        binsof(B_cp1) intersect{MAXPOS, MAXNEG, ZERO};
}

opcode_cin_cs : cross opcode_cp, cin_cp{
    option.cross_auto_bin_max = 0;
    bins opcode_cin = binsof(opcode_cp) intersect{ADD} &&
                      binsof(cin_cp);
}

opcode_dir_cs : cross opcode_cp, dir_cp{
    option.cross_auto_bin_max = 0;
    bins opcode_dir = binsof(opcode_cp) intersect{SHIFT, ROTATE} &&
                      binsof(dir_cp);
}

opcode_shift_cs : cross opcode_cp, shift_cp{
    option.cross_auto_bin_max = 0;
    bins opcode_shift = binsof(opcode_cp) intersect{SHIFT} &&
                        binsof(shift_cp);
}
```

```
opcode_redA_walkA_cs : cross opcode_cp, redA_cp, A_cp2, B_cp1{
    option.cross_auto_bin_max = 0;
    bins opcode_redA_walkA = binsof(opcode_cp) intersect{OR,XOR} &&
                             binsof(redA_cp.redA_on) &&
                             binsof(A_cp2) &&
                             binsof(B_cp1.B_data_0);
}

opcode_redB_walkB_cs : cross opcode_cp, redB_cp, B_cp2, A_cp1{
    option.cross_auto_bin_max = 0;
    bins opcode_redB_walkB = binsof(opcode_cp) intersect{OR,XOR} &&
                             binsof(redB_cp.redB_on) &&
                             binsof(B_cp2) &&
                             binsof(A_cp1.A_data_0);
}

invalid_case_cs : cross opcode_cp, redA_cp, redB_cp{
    option.cross_auto_bin_max = 0;
    bins invalid_case = !binsof (opcode_cp) intersect{OR,XOR} &&
                        (binsof(redB_cp.redB_on) || binsof(redA_cp.redA_on));
}

endgroup
```

```
        function new(string name = "ALSU_coverage", uvm_component parent = null);
            super.new(name,parent);
            // create covergroup
            cvr_gp = new();
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            cov_export = new("cov_export",this);
            cov_fifo = new("cov_fifo",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            cov_export.connect(cov_fifo.analysis_export);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                cov_fifo.get(seq_item_cov);
                // sample covergroup
                cvr_gp.sample();
            end
        endtask



    endclass


endpackage
```

```
module ALSU_sva(ALSU_if vif);

    property reset_check;
        @(posedge vif.clk)  vif.rst |=> (vif.leds == 0 && vif.out == 0);
    endproperty
    assert_reset: assert property(reset_check);
    cover_reset: cover property(reset_check);

    property invalid_leds_out;
        @(posedge vif.clk) disable iff (vif.rst)
        ( ( ((vif.red_op_A) | (vif.red_op_B)) & ((vif.opcode[1]) | (vif.opcode[2])) )
          | ((vif.opcode[1]) & (vif.opcode[2])) )
        |=> ##1 (vif.leds == ~$past(vif.leds));
    endproperty

    assert_invalid_led: assert property(invalid_leds_out) else $display("Invalid input combination not detected at time %0t: red_op_A = %0b, red_
    cover_invalid_led: cover property(invalid_leds_out);

    property invalid_out;
        @(posedge vif.clk) disable iff (vif.rst)
        ( ( ((vif.red_op_A) | (vif.red_op_B)) & ((vif.opcode[1]) | (vif.opcode[2])) )
          | ((vif.opcode[1]) & (vif.opcode[2])) )
        |-> ##1 (vif.out == 0);
    endproperty

    assert_invalid_out: assert property(invalid_out) else $display("Invalid input combination not detected at time %0t: red_op_A = %0b, red_op_B =
    cover_invalid_out: cover property(invalid_out);

endmodule
```

/ALSU_coverage_pkg/ALSU_coverage	100.00%				
TYPE cvr_gp	100.00%	100	100.00...		auto(0)
CVP cvr_gp::A_cp1	100.00%	100	100.00...		
CVP cvr_gp::A_cp2	100.00%	100	100.00...		
CVP cvr_gp::B_cp1	100.00%	100	100.00...		
CVP cvr_gp::B_cp2	100.00%	100	100.00...		
CVP cvr_gp::ALU_cp	100.00%	100	100.00...		
CVP cvr_gp::opcode_cp	100.00%	100	100.00...		
CVP cvr_gp::cin_cp	100.00%	100	100.00...		
CVP cvr_gp::dir_cp	100.00%	100	100.00...		
CVP cvr_gp::shift_cp	100.00%	100	100.00...		
CVP cvr_gp::redA_cp	100.00%	100	100.00...		
CVP cvr_gp::redB_cp	100.00%	100	100.00...		
CROSS cvr_gp::add_mult_AB_cs	100.00%	100	100.00...		
CROSS cvr_gp::opcode_cin_cs	100.00%	100	100.00...		
CROSS cvr_gp::opcode_dir_cs	100.00%	100	100.00...		
CROSS cvr_gp::opcode_shift_cs	100.00%	100	100.00...		
CROSS cvr_gp::opcode_redA_walkA_cs	100.00%	100	100.00...		
CROSS cvr_gp::opcode_redB_walkB_cs	100.00%	100	100.00...		
CROSS cvr_gp::invalid_case_cs	100.00%	100	100.00...		
INST \/ALSU_coverage_pkg::ALSU_coverage::cvr_gp	100.00%	100	100.00...		0
CVP A_cp1	100.00%	100	100.00...		
CVP A_cp2	100.00%	100	100.00...		
CVP B_cp1	100.00%	100	100.00...		
CVP B_cp2	100.00%	100	100.00...		
CVP ALU_cp	100.00%	100	100.00...		
CVP opcode_cp	100.00%	100	100.00...		
CVP cin_cp	100.00%	100	100.00...		
CVP dir_cp	100.00%	100	100.00...		
CVP shift_cp	100.00%	100	100.00...		
CVP redA_cp	100.00%	100	100.00...		
CVP redB_cp	100.00%	100	100.00...		
CROSS add_mult_AB_cs	100.00%	100	100.00...		
CROSS opcode_cin_cs	100.00%	100	100.00...		
CROSS opcode_dir_cs	100.00%	100	100.00...		
CROSS opcode_shift_cs	100.00%	100	100.00...		
CROSS opcode_redA_walkA_cs	100.00%	100	100.00...		
CROSS opcode_redB_walkB_cs	100.00%	100	100.00...		
CROSS invalid_case_cs	100.00%	100	100.00...